

Java - Technical Interview Exercise

Project Overview

The objective of this exercise is the creation of a RESTful API leveraging Java and Spring Boot. You are expected to employ Clean Architecture and TDD methodologies to build a robust solution backed by a reliable data store.

Implementation must be guided by the specific Pokemon user stories detailed hereafter; these requirements should be clearly integrated into your final interview presentation.

This service will interface with the PokeAPI to facilitate data retrieval, local replication, and attribute modification for various Pokemon entries.

Refer to the official PokeAPI documentation for integration details:

<https://pokeapi.co/docs/v2>

Requirements

Functional Requirements

- **PokeAPI Integration**
 - Construct a RESTful API using Spring Boot that communicates with the external PokeAPI.
- **User Story 01: Pokemon Enumeration**
 - The system should allow users to browse Pokemon via paginated results, displaying each entry's sprite, category, mass, and a collection of their skills.
 - Nice to have: Implement caching for service responses.
- **User Story 02: Detailed View**
 - Users must be able to access comprehensive data for a chosen Pokemon, specifically viewing its image, core statistics, narrative description, and evolutionary lineage.
- **User Story 03: Data Synchronization**
 - Develop a mechanism to persist Pokemon data into a local relational store.
 - This replication layer is intended to facilitate the addition of proprietary fields.
 - Use cases include localized nomenclature, geographical metadata, or internal classification tags.
- **User Story 04: Local Data Modification**
 - Enable update operations for any Pokemon currently stored within the local database.
 - Ensure robust validation: provide 404 responses for missing records, 400 status codes for malformed payloads, and incorporate further defensive logic as required.

Java - Technical Interview Exercise

Technical Requirements

- **Mandatory**
 - Host the code in a public Git repository
 - Include tests
 - Proper error handling
 - Nice to have: Implement a caching layer for PokeAPI responses
 - Front-end that consumes the API (language of your choice)
- **Optional**
 - Any additional functionality is welcome
- **Database**
 - Establish a relational database or appropriate data storage solution containing a primary entity and a secondary collection for user management.
 - Records must include a unique primary key and a minimum of two descriptive attributes.
- **API**
 - Construct an Java Web API facilitating comprehensive CRUD operations for the defined dataset.
 - Ensure all endpoints utilize standard HTTP verbs, required parameters, and consistent return structures.
 - Implement an auxiliary API for user registration, authentication, and the management of protected versus public routes.
- **Data Layer**
 - Design a specialized data access layer to manage interactions with the persistence store, providing the foundational logic for the API controllers.
- **Core Business Logic**
 - Develop a dedicated business logic layer to encapsulate all domain rules and data validation procedures.
 - This layer should maintain architectural independence from both the API and the data access components.
- **Testing and Validation**
 - Provide thorough unit test coverage for every core component within the application suite.

Frontend

- While the primary emphasis resides in backend proficiency, a comprehensive demonstration of **full-stack development capabilities** is required. Consequently:
 - Integrate your established backend service with a modern frontend framework of your choosing (e.g., React or Vue).
 - Primary Evaluation Criteria:
 - The interface must exhibit responsiveness and a user-centric design.

Java - Technical Interview Exercise

- Execute standard CRUD operations corresponding to the defined functional use cases.
- Architectural integrity: Maintain clean component organization and efficient state management.
- Submission Guidelines:
 - Provide a comprehensive **README** detailing environment configuration and pertinent technical documentation.
 - The application should be pre-populated with seeded data or mock credentials for demonstration purposes.

Delivery

- Provide a README file containing environment setup procedures and technical documentation.
- The solution must be pre-populated with relevant demonstration data.
- Supply a Dockerfile for containerized execution.

Generative AI tools

Imagine you're tasked with generating a **RESTful API** for a simple **task management system** using your preferred language. The system should support the following functionality:

- Create, read, update, and delete tasks (CRUD)
- Each task has a **title**, **description**, **status**, and **due_date**
- Tasks are associated with a user (assume basic User model exists)

Instructions:

- Using your preferred **GenAI coding tool** (e.g., Cursor, Claude Code, Windsurf, GitHub Copilot, etc.), write the **prompt you would use** to generate the API scaffold or full implementation.
- Show the **output code** (or a representative sample of it).
- Describe how you:
 - Validated the AI's suggestions
 - Corrected or improved the output, if necessary
 - Handled edge cases, authentication, or validations

Presentation and Code Review

You will be required to present your project to the technical interview panel. During the presentation, you should explain your user story, design choices, the technical architecture, and demonstrate the functionality of the application. This will be done over Google Meets or Zoom

Java - Technical Interview Exercise

and you will screen share either your GitHub repository or IDE.

After the presentation, the interview panel will conduct a code review of your project. You will be asked to explain your coding decisions and answer any questions related to the code. The interview panel will evaluate your project based on the following criteria:

- **Clean Architecture:** Your architecture should adhere to Clean Architecture principles, including separation of concerns and independence of components.
- **Application testing:** Your project should have sufficient test coverage. Use of TDD is preferable.
- **Code quality:** Your code should be well-organized, readable, and adhere to best practices.
- **Functionality:** Your application should perform as expected in the requirements without errors or bugs. Optional but desired: no warnings in the browser console.
- **Presentation:** Your presentation should be clear, concise, and demonstrate a good understanding of the project and of the main **backend** and **frontend** best practices.
- **GenAI tools:** Your answers and presentation must show fluency with GenAI tools and prompt engineering, and critical thinking when evaluating AI-generated code.